AGENDA 10

Funções

SEAD- Superintendência de Ensino e Pesquisa nas modalidades EaD e Aberta

GOVERNO DO ESTADO DE SÃO PAULO

EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

DESENVOLVIMENTO DE SISTEMAS I

Expediente

Autores:

PAULO EDUARDO CARDOSO ANDRADE

TATIANE TOLENTINO DE ASSIS

Revisão Técnica:
Kelly Cristiane de Oliveira Dal Pozzo

São Paulo – SP, 2025

MOMENTO REFLEXÃO



No dia a dia, muitas tarefas se repetem. Damos as mesmas orientações, seguimos rotinas parecidas, refazemos caminhos que já conhecemos. Com o tempo, aprendemos que repetir não significa fazer sempre do zero, mas organizar melhor aquilo que já sabemos.

Quando criamos um hábito, um método ou uma rotina bem definida, ganhamos clareza, economia de tempo e segurança. Passamos a confiar no processo e conseguimos dedicar mais atenção ao que realmente importa.

Para refletir:

- O que acontece quando tudo é feito sem organização?
- Como sua rotina muda quando cada tarefa tem um “lugar” e um “propósito”?
- Será que repetir pode ser algo positivo quando existe método?



Aprender a estruturar ações de forma consciente é um passo importante para qualquer área do conhecimento inclusive na tecnologia.

POR QUE APRENDER?



Em contextos educacionais, profissionais e organizacionais, a tecnologia é utilizada para resolver problemas que se repetem, evoluem e se tornam cada vez mais complexos. À medida que uma solução cresce, torna-se inviável mantê-la apenas com ações isoladas ou improvisadas. Surge, então, a necessidade de estruturar o pensamento, planejar etapas e organizar responsabilidades dentro de um sistema.

Aprender este conceito permite transformar soluções simples em estruturas mais claras, reutilizáveis e sustentáveis ao longo do tempo. Em vez de refazer constantemente a mesma tarefa, o estudante passa a adotar uma lógica de reaproveitamento, o que reduz erros, economiza tempo e melhora a qualidade do resultado final.

Do ponto de vista técnico, essa abordagem favorece:

- A padronização de procedimentos;
- A manutenção e a evolução dos sistemas;
- A leitura e compreensão por outras pessoas;
- O trabalho colaborativo, comum em ambientes profissionais.

Além disso, desenvolver essa habilidade contribui diretamente para o amadurecimento do raciocínio lógico e da capacidade de análise, pois o estudante aprende a identificar padrões, separar responsabilidades e compreender que grandes problemas são resolvidos a partir da combinação de pequenas partes bem definidas.

Portanto, aprender este tema não se limita ao domínio de uma linguagem ou ferramenta específica. Trata-se de adquirir uma forma de pensar que será aplicada em diferentes situações, projetos e desafios ao longo da formação acadêmica e da atuação profissional.

PARA COMEÇAR O ASSUNTO

Ao desenvolver um programa, muitas vezes é necessário repetir o mesmo conjunto de instruções em diferentes momentos. Quando essas instruções são escritas repetidamente, o código se torna maior, mais confuso e difícil de manter.



Para resolver esse problema, a programação utiliza o conceito de função.

Uma função é um trecho de código que executa uma tarefa específica e recebe um nome. Esse trecho pode ser utilizado sempre que necessário, bastando chamá-lo pelo nome definido. Dessa forma, o programador evita repetições e organiza melhor a lógica do programa.

MERGULHANDO NO TEMA

Em programação, uma função pode ser compreendida como um bloco de código criado para executar uma tarefa específica dentro de um programa. Em vez de escrever instruções repetidas ao longo do código, o desenvolvedor define uma função com um nome e a reutiliza sempre que aquela ação for necessária.

Essa estrutura não é apenas uma questão de organização estética: ela reflete diretamente a forma como o programador pensa e resolve problemas. Ao dividir um programa em funções, você transforma uma tarefa complexa em partes menores e mais controláveis. Por exemplo, em um sistema de cadastro de usuários, uma função pode ser responsável apenas por validar os dados, enquanto outra cuida do armazenamento dessas informações.

Um dos principais benefícios do uso de funções está na reutilização de código. Imagine um sistema em

que seja necessário calcular o valor total de uma compra diversas vezes. Sem funções, o mesmo conjunto de instruções precisaria ser repetido em vários pontos do programa. Isso aumenta o risco de erros e dificulta alterações futuras.

Ao utilizar funções, o desenvolvedor cria uma única solução para esse problema e a reutiliza sempre que necessário. Isso impacta diretamente a manutenção do sistema. Se houver necessidade de alteração na lógica, basta modificar a função uma única vez, e todo o programa será atualizado automaticamente.

Esse aspecto é essencial no mercado de trabalho, especialmente em equipes que utilizam ferramentas de versionamento como o Git. Em projetos colaborativos, funções bem definidas permitem que diferentes desenvolvedores trabalhem em partes específicas do sistema sem comprometer o funcionamento geral.

Uma forma bastante didática de entender o funcionamento de uma função é compará-la a uma máquina. Nessa analogia, você fornece dados de entrada, a função realiza um processamento interno e, ao final, retorna um resultado.

Na prática, essa estrutura corresponde aos parâmetros (entrada), ao conjunto de instruções dentro da função (processamento) e ao valor retornado (saída). Por exemplo, uma função pode receber dois números, realizar uma operação matemática e devolver o resultado dessa operação.

A linguagem Python já oferece diversas funções prontas que facilitam o desenvolvimento de programas. Entre as mais utilizadas, estão aquelas responsáveis por exibir informações na tela, receber dados do usuário, calcular o tamanho de estruturas e realizar conversões de tipos.

No entanto, mais importante do que conhecer essas funções é entender quando e por que utilizá-las. Em um sistema real, por exemplo, a função de entrada de dados pode ser usada para coletar informações de um usuário em um formulário, enquanto funções de conversão garantem que esses dados estejam no formato correto para processamento.

Um erro comum entre iniciantes é utilizar essas funções de forma mecânica, sem compreender seu papel na lógica do programa. Isso pode levar a falhas, como tratar dados de forma inadequada ou gerar resultados inesperados.

O uso adequado de funções contribui diretamente para a qualidade do software desenvolvido. Programas estruturados em funções tendem a ser mais legíveis, mais fáceis de corrigir e mais simples de expandir com novas funcionalidades.

Por outro lado, a ausência desse recurso pode gerar códigos longos, repetitivos e difíceis de entender. Em um ambiente profissional, isso aumenta o tempo de desenvolvimento, dificulta o trabalho em equipe e eleva o risco de erros críticos.



Pense em uma função como uma máquina:
você coloca dados (entrada), ela processa, e devolve um resultado (saída).

Estrutura básica de uma função em Python

Em Python, uma função é criada utilizando a palavra-chave **def**, seguida do nome da função, parênteses e dois pontos. O conjunto de instruções que pertence à função deve estar indentado, ou seja, recuado em relação à margem esquerda. A estrutura básica de uma função pode ser representada da seguinte forma:

```python
def nome_da_funcao():
    # instruções da função
```

Cada parte dessa estrutura possui uma função específica:

- **Def -** indica ao Python que uma função está sendo definida;
- **nome_da_funcao -** identifica a função e deve representar claramente sua finalidade;
- **parênteses ( ) -** indicam onde podem ser definidos parâmetros (quando houver);
- **dois pontos : -** sinalizam o início do bloco da função;
- **indentação -** define quais instruções pertencem à função.

Vamos visualizar um exemplo de função que simula uma situação comum em sistemas: a exibição de uma mensagem de erro padrão.

```python
def erro_acesso():
    print("Acesso negado. Verifique suas credenciais.")
```

No exemplo, a função erro_acesso possui uma tarefa bem definida: exibir uma mensagem de erro ao usuário. Esse tipo de função é bastante utilizado em sistemas reais para padronizar mensagens e evitar repetições no código. Para que a função seja executada, é necessário chamá-la pelo nome.

```python
erro_acesso()
```

O resultado será:



Definir uma função não significa que ela será executada automaticamente. A função apenas fica disponível no programa e só será executada quando for chamada explicitamente.

Funções sem e com parâmetros

As funções podem ser classificadas de acordo com a forma como recebem informações para executar suas tarefas. Em Python, é bastante comum trabalhar com dois tipos principais: funções que não recebem parâmetros e funções que recebem parâmetros.

Essa distinção não é apenas teórica. Ela influencia diretamente a forma como o programa se comporta e o nível de flexibilidade que o desenvolvedor consegue alcançar ao construir soluções computacionais.

Funções sem parâmetros

As funções sem parâmetros são aquelas que não recebem informações externas. Elas executam sempre a mesma ação, independentemente do contexto em que são chamadas.

Esse tipo de função é indicado quando:

- A tarefa é fixa;
- A ação não depende de dados fornecidos pelo usuário;
- É necessário apenas executar um procedimento padrão.

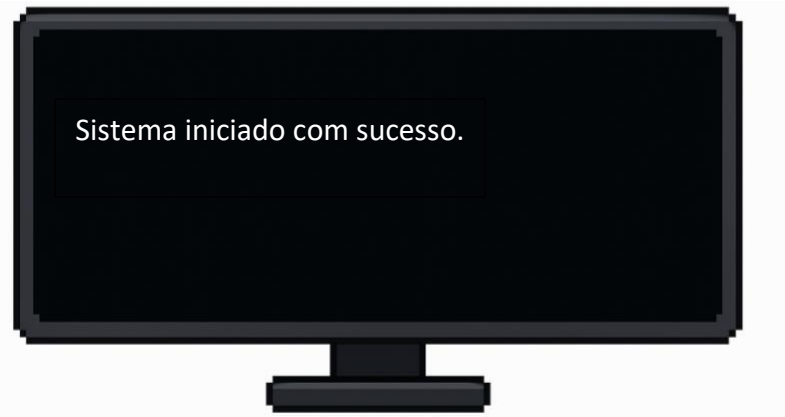
Exemplo:

```python
def sistema_iniciado():
    print("Sistema iniciado com sucesso.")
```

Ao chamar essa função:

```python
sistema_iniciado()
```

A mensagem exibida será sempre a mesma.



Funções com parâmetros

As funções com parâmetros recebem informações externas que influenciam o seu funcionamento. Esses parâmetros funcionam como entradas que permitem que a mesma função seja utilizada em diferentes situações.

Esse tipo de função é indicado quando:

- A tarefa depende de dados variados;
- É necessário reutilizar a mesma lógica com valores diferentes;
- O sistema precisa ser mais flexível.

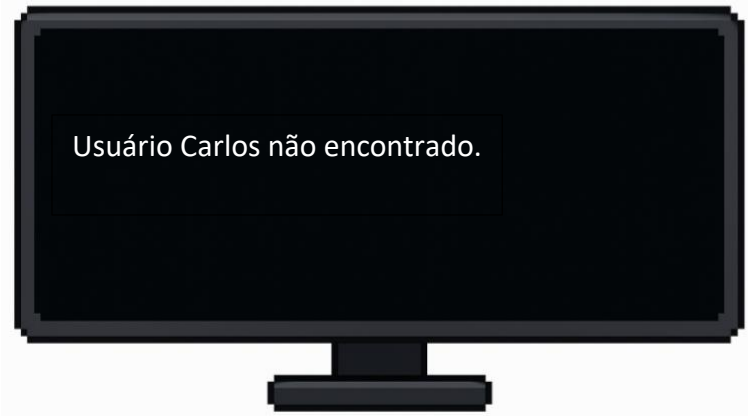
Exemplo:

```python
def erro_usuario(nome):
    print(f"Usuário {nome} não encontrado.")
```

Ao chamar essa função:

```python
erro_usuario("Carlos")
```

A mensagem exibida será adaptada de acordo com o dado fornecido.



Mas o que são parâmetros?

Parâmetros são variáveis declaradas na definição de uma função e têm a finalidade de receber informações externas no momento em que a função é utilizada. Eles permitem que uma mesma função execute a mesma lógica, mas produza resultados diferentes de acordo com os dados fornecidos.

Quando a função é chamada, os valores enviados para esses parâmetros recebem o nome de argumentos. Ou seja, o parâmetro representa o “espaço” que irá receber um valor, enquanto o argumento é o valor efetivamente enviado para esse espaço no momento da execução.

De forma simplificada:

- O parâmetro é definido dentro da função e indica quais dados ela espera receber;
- O argumento é o valor real informado quando a função é chamada.

Essa distinção é importante porque permite compreender como as funções se tornam flexíveis e reutilizáveis, adaptando-se a diferentes situações sem a necessidade de criar várias funções semelhantes.

Parâmetro: variável definida na função para receber dados;

Argumento: valor passado para o parâmetro quando a função é executada.

Com o uso adequado de parâmetros, as funções deixam de ser estáticas e passam a responder de forma dinâmica às necessidades do programa.

Comparação entre funções sem e com parâmetros

Característica	Sem parâmetros	Com parâmetros
Recebem dados externos	Não	Sim
Resultado variável	Não	Sim
Reutilização	Limitada	Maior
Flexibilidade	Baixa	Alta
Uso comum	Mensagens fixas, inicialização	Mensagens personalizadas, cálculos, validações

Ao compreender a diferença entre esses dois tipos de função, o estudante passa a escolher de forma mais consciente qual estrutura utilizar em cada situação, tornando seus programas mais organizados, reutilizáveis e eficientes.

Funções com retorno (return)

Até o momento, foram apresentadas funções que executam ações, como exibir mensagens ou registrar operações. No entanto, em muitos casos, uma função precisa produzir um resultado para que ele seja utilizado em outras partes do programa. Para isso, utilizamos o comando return.

Uma função com retorno é aquela que, após executar seu processamento, devolve um valor ao programa que a chamou. Esse valor pode ser armazenado em uma variável, utilizado em cálculos ou em decisões posteriores.

Diferença entre exibir e retornar um valor

Ao trabalhar com funções, é fundamental compreender que exibir um valor e retornar um valor são ações diferentes, com finalidades distintas dentro de um programa.

Quando uma função exibe um valor, ela apenas mostra a informação na tela naquele momento. Essa ação é útil para informar o usuário, mas o valor exibido não fica disponível para o restante do programa. Ou seja, após a exibição, o programa não consegue reutilizar essa informação para outros cálculos ou decisões.

Já quando uma função retorna um valor, ela envia o resultado do seu processamento de volta para o ponto onde foi chamada. Esse valor pode ser armazenado em uma variável, utilizado em expressões, comparações ou em outras funções. Dessa forma, o retorno permite que o resultado continue sendo usado ao longo do programa.

O comando return tem duas funções importantes:

- Encerrar a execução da função;
- Devolver um valor ao programa que fez a chamada.

A partir do momento em que o return é executado, a função é finalizada imediatamente, e o valor retornado passa a fazer parte do fluxo do programa.



Dica:

- Exibir serve para informar;
- Retornar serve para reutilizar;

Programas mais completos dependem do uso de funções que retornam valores.

Compreender essa diferença é essencial para avançar na criação de sistemas que não apenas mostram resultados, mas que também tomam decisões e processam dados de forma estruturada.

Exemplo:

Imagine um sistema de escritório que calcula o total de horas trabalhadas em um dia. Esse valor precisa ser utilizado posteriormente para relatórios ou cálculos de pagamento, portanto deve ser retornado.

```python
def calcular_horas(entrada, saida):
    total = saida - entrada
    return total
```

Ao chamar a função, o valor retornado pode ser armazenado:

```python
horas_trabalhadas = calcular_horas(8, 17)
```

Nesse caso, a função é estruturada para receber informações externas, representadas pelos valores de entrada e saída. Esses dados são utilizados internamente para realizar o processamento necessário, no caso, um cálculo.

Após concluir esse processamento, a função retorna o resultado, em vez de apenas exibi-lo. Isso significa que o valor produzido não se perde após a execução da função: ele é devolvido ao programa que fez a chamada.

Dessa forma, o valor retornado:

- Pode ser armazenado em uma variável;
- Pode ser reutilizado em outros cálculos;
- Pode servir de base para decisões e verificações posteriores.

Esse comportamento torna a função mais útil e flexível, pois o resultado passa a integrar o fluxo do programa, permitindo que outras partes do sistema façam uso da informação gerada.

Uso do valor retornado

O valor devolvido por uma função não se limita a um único uso. Como ele retorna ao fluxo principal do programa, pode ser reaproveitado de diversas formas, ampliando as possibilidades de processamento e tomada de decisão. Esse valor pode ser armazenado para uso posterior, exibido ao usuário, utilizado como base para verificações lógicas ou ainda integrado a novos cálculos, tornando o programa mais dinâmico e eficiente.

O valor devolvido por uma função pode ser utilizado de diferentes formas, como:

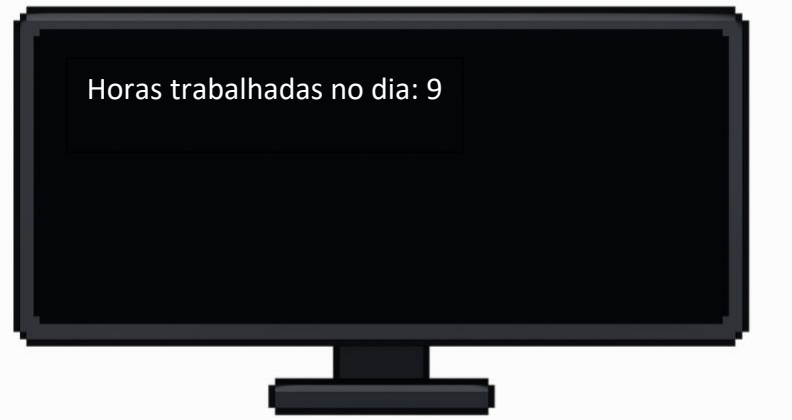
- Atribuição a uma variável;
- Exibição em tela;
- Uso em decisões (condições);
- Parte de outros cálculos.

Essa flexibilidade é o que torna as funções com retorno um dos recursos mais importantes na construção de programas estruturados e reutilizáveis.

Exemplo:

```python
print(f"Horas trabalhadas no dia: {calcular_horas(8,17)}")
```

Esse comando reúne duas ações em uma única linha: a chamada de uma função com retorno e a exibição do resultado na tela.



Primeiramente, a função calcular_horas(8,17) é executada. Os valores 8 e 17 são passados como argumentos para a função, que realiza o cálculo interno e retorna o total de horas trabalhadas no dia.

Em seguida, o valor retornado pela função é inserido automaticamente dentro da mensagem exibida, por meio da f-string (string formatada). Tudo o que está entre chaves { } é avaliado antes da impressão, e o resultado é substituído no texto final.

Por fim, o comando print exibe a mensagem completa na tela, já com o valor calculado incorporado ao texto.

Em resumo, esse código:

- Chama uma função que retorna um valor;
- Utiliza esse valor diretamente dentro de uma mensagem;
- Exibe o resultado final ao usuário em uma única instrução.

Essa forma de escrita é comum em programas profissionais, pois torna o código mais direto e legível, evitando linhas desnecessárias.



Ao executar o return, a função é finalizada imediatamente. Funções com retorno tornam o programa mais flexível e organizado.

O uso de funções com retorno representa um avanço significativo na forma de construir programas. Ao invés de apenas executar ações pontuais ou exibir informações na tela, o programa passa a produzir resultados que integram o fluxo lógico da aplicação.

Esses resultados podem ser armazenados, analisados, combinados com outros dados e utilizados em decisões futuras, tornando o sistema mais organizado, flexível e eficiente. Com isso, o programador deixa de pensar apenas em comandos isolados e passa a estruturar soluções de forma mais estratégica.

Compreender e aplicar funções com retorno é fundamental para o desenvolvimento de programas mais robustos, reutilizáveis e alinhados às práticas utilizadas em aplicações reais e profissionais.

Boas práticas no uso de funções

Além de saber criar e utilizar funções, é importante adotar boas práticas que tornam o código mais organizado, legível e fácil de manter. Essas práticas aproximam o estudante da forma como os programas são desenvolvidos em ambientes profissionais.

Utilize nomes claros e objetivos: O nome de uma função deve indicar claramente o que ela faz. Nomes bem escolhidos facilitam a leitura do código e ajudam outras pessoas a compreenderem sua finalidade sem a necessidade de explicações adicionais. Evite nomes genéricos e prefira termos que representem a ação realizada.

Cada função deve ter uma única responsabilidade: Uma função deve executar apenas uma tarefa específica. Quando uma função faz muitas coisas ao mesmo tempo, ela se torna difícil de entender e de manter. Dividir o problema em funções menores torna o programa mais organizado e reduz a chance de erros.

Evite funções muito longas: Funções extensas indicam que o código pode estar mal organizado. Sempre que uma função começar a ficar grande demais, avalie a possibilidade de separá-la em funções menores e mais específicas. Funções curtas são mais fáceis de testar, corrigir e reutilizar.

Use parâmetros apenas quando necessário: Parâmetros tornam as funções mais flexíveis, mas devem ser usados com critério. Utilize parâmetros apenas quando a função realmente precisar receber informações externas para cumprir sua tarefa. Isso evita funções confusas e com responsabilidades mal definidas.

Prefira funções com retorno quando o valor precisar ser reutilizado: Sempre que o resultado de um processamento for utilizado em outras partes do programa, opte por funções que retornem valores. Isso torna o código mais versátil e evita dependência de exibições diretas na tela.

Mantenha consistência no estilo do código: Adotar um padrão de escrita — nomes, espaçamentos e organização — torna o código mais profissional e facilita o trabalho em equipe. A consistência ajuda tanto quem escreve quanto quem lê o programa.

Boas práticas no uso de funções

Situação	✔ Adequado	✘ Inadequado
Nome da função	Nome claro que indica a ação	Nome genérico ou sem sentido
Responsabilidade	Executa apenas uma tarefa	Faz várias coisas ao mesmo tempo
Tamanho da função	Curta e objetiva	Muito longa e confusa
Uso de parâmetros	Apenas quando necessário	Parâmetros em excesso ou desnecessários
Uso de retorno	Retorna valores que serão reutilizados	Apenas exibe valores quando poderia retornar
Organização	Código legível e padronizado	Código desorganizado e difícil de entender

Antes de considerar finalizada sua função, utilize o checklist de boas práticas sugerido a seguir para revisar suas funções. Essa revisão ajuda a identificar ajustes simples que podem tornar o código mais claro, eficiente e alinhado às boas práticas de programação.

Checklist — Minha função segue boas práticas?

Antes de considerar uma função finalizada, verifique:

- ✓ O nome da função indica claramente o que ela faz?
- ✓ A função executa apenas uma tarefa?
- ✓ O tamanho da função está adequado e fácil de entender?
- ✓ Os parâmetros são realmente necessários?
- ✓ O uso de return está correto quando o valor precisa ser reutilizado?
- ✓ O código está organizado e legível?

Seguir boas práticas no uso de funções vai além de escrever códigos que simplesmente funcionam. Essas orientações contribuem para a construção de programas mais claros, organizados e eficientes, capazes de evoluir ao longo do tempo sem comprometer sua compreensão ou manutenção.

Vamos programar!

Módulo de faturamento SaaS

Você está desenvolvendo um módulo básico de faturamento para uma empresa SaaS.

O sistema trabalha com códigos internos de plano e precisa calcular o valor final da fatura, considerando apenas:

- plano contratado (via código);
- tipo de cliente (PF ou PJ);
- imposto aplicado conforme o tipo de cliente.

Para facilitar manutenção e evolução futura do sistema, cada regra será implementada em uma função específica, seguindo boas práticas.

Objetivos

- Criar funções pequenas e coesas;
- Trabalhar com códigos internos (padrão de mercado);
- Utilizar funções com retorno simples;
- Separar regras de negócio em funções independentes.

Etapa 1 — Função para identificar o nome do plano

Situação:

Os planos são armazenados no sistema por códigos numéricos, mas o nome amigável precisa ser exibido ao usuário.

python

```python
def nome_plano(cod_plano):
    if cod_plano == 101:
        return "Básico"
    elif cod_plano == 202:
        return "Pro"
    elif cod_plano == 303:
        return "Enterprise"
    else:
        return "Plano inválido"
```

Essa função tem como objetivo interpretar um código interno do sistema e convertê-lo em uma informação legível para o usuário. Cada estrutura if é responsável por verificar se o código informado corresponde a um plano específico. Quando a condição é atendida, a função retorna imediatamente o nome do plano associado àquele

código. Caso nenhuma das condições seja satisfeita, o else trata a situação como inválida, garantindo que o sistema tenha uma resposta padrão para códigos não reconhecidos.

Como alternativa, essa função poderia ser implementada de forma mais elegante, atribuindo o resultado a uma variável ao longo das condições e realizando apenas um único return ao final da função. Essa abordagem melhora a legibilidade do código e facilita futuras manutenções, pois centraliza o ponto de retorno e torna o fluxo da função mais claro.

python

```python
def nome_plano(cod_plano):
    nome = "Plano inválido"

    if cod_plano == 101:
        nome = "Básico"
    elif cod_plano == 202:
        nome = "Pro"
    elif cod_plano == 303:
        nome = "Enterprise"

    return nome
```

Nesta versão, o resultado é armazenado em uma variável ao longo das verificações, e o retorno ocorre apenas ao final da função.



Ambas as versões são corretas e funcionais. A escolha entre elas depende do nível de experiência do programador e do padrão adotado no projeto. Para fins didáticos, a primeira versão ajuda a compreender a lógica condicional; já a segunda aproxima de práticas mais comuns em ambientes profissionais.

Etapa 2 — Função para obter o valor base do plano

Cada plano possui um valor base fixo, definido a partir do código.

```python
def valor_base_plano(cod_plano):
    if cod_plano == 101:
        return 59.90
    elif cod_plano == 202:
        return 99.90
    elif cod_plano == 303:
        return 199.90
    else:
        return None
```

Essa função tem como objetivo determinar o valor base de um plano a partir de um código interno, prática comum em sistemas reais para padronizar e facilitar o controle das informações. O parâmetro cod_plano representa o identificador do plano contratado e é utilizado como base para as verificações internas.

A estrutura condicional avalia, por meio de if e elif, se o código informado corresponde a um dos planos previamente definidos. Cada condição associa um código específico a um valor monetário fixo, que representa o custo base daquele plano. Assim que uma condição é atendida, a função retorna imediatamente o valor correspondente, encerrando sua execução.

Caso o código informado não corresponda a nenhum dos planos válidos, o bloco else é acionado e a função retorna None. Esse retorno indica, de forma controlada, que o código recebido é inválido ou desconhecido, permitindo que o programa principal trate essa situação adequadamente, evitando cálculos incorretos ou comportamentos inesperados.

Dessa forma, a função centraliza a regra de definição de valores dos planos, tornando o código mais organizado, legível e fácil de manter.

Etapa 3 — Função para cálculo de imposto

O imposto aplicado varia conforme o tipo de cliente:

- Pessoa Física (PF): 5%

- Pessoa Jurídica (PJ): 9%

```python
def calcular_imposto(valor, tipo_cliente):
    tipo_cliente = tipo_cliente.upper()

    if tipo_cliente == "PF":
        return valor * 0.05
    elif tipo_cliente == "PJ":
        return valor * 0.09
    else:
        return None
```

Essa função tem como finalidade centralizar a regra de cálculo de imposto aplicada sobre um determinado valor, isolando essa responsabilidade do restante do sistema. O parâmetro valor representa a base sobre a qual o imposto será calculado, enquanto tipo_cliente indica a categoria do cliente, que influencia diretamente a alíquota utilizada.

Inicialmente, o valor de tipo_cliente é convertido para letras maiúsculas por meio do método upper(). Essa etapa garante maior robustez ao código, permitindo que a função trate corretamente diferentes formas de entrada, como "pf", "Pf" ou "PF".

A estrutura condicional verifica, então, o tipo de cliente informado. Caso seja "PF", a função aplica uma alíquota de 5% sobre o valor recebido. Se o tipo for "PJ", aplica-se uma alíquota de 9%. Em ambos os casos, o resultado do cálculo é retornado imediatamente.

Se o tipo de cliente não corresponder a nenhuma das opções válidas, a função retorna None. Esse retorno sinaliza de forma simples e controlada que a informação fornecida é inválida, permitindo que o programa principal trate essa situação de maneira adequada.

Ao concentrar toda a lógica de imposto em uma única função, o código se torna mais organizado e flexível. Caso as regras fiscais sejam alteradas no futuro, basta modificar essa função, sem a necessidade de revisar outras partes do sistema.

Etapa 4 — Função principal: cálculo do total da fatura

Agora o sistema precisa calcular o valor final da fatura, somando:

- valor base do plano;

- imposto correspondente ao tipo de cliente.

```python
def calcular_total_fatura(cod_plano, tipo_cliente):
    valor_base = valor_base_plano(cod_plano)
    if valor_base is None:
        return None

    imposto = calcular_imposto(valor_base, tipo_cliente)
    if imposto is None:
        return None

    total = valor_base + imposto
    return total
```

Essa função atua como função coordenadora do processo de faturamento, sendo responsável por organizar o fluxo de execução e integrar as demais funções do sistema. Em vez de concentrar regras específicas, ela utiliza funções menores e reutilizáveis para obter cada parte do cálculo, o que torna o código mais modular e fácil de manter.

Inicialmente, a função solicita o valor base do plano por meio da chamada à função valor_base_plano. Caso o código do plano seja inválido, a função retorna None, interrompendo o processo de forma controlada e evitando cálculos incorretos.

Em seguida, a função chama calcular_imposto, passando o valor base e o tipo de cliente. Novamente, se a informação fornecida for inválida, o retorno None é utilizado como sinalização para encerrar a execução de maneira segura.

Somente após validar todas as etapas anteriores é que o cálculo final é realizado, somando o valor base ao imposto correspondente. O resultado é então retornado ao programa principal.

Etapa 5 — Programa principal (uso do sistema)

O programa principal utiliza as funções criadas para gerar a fatura e exibir as informações ao usuário.

python

```
# Dados de entrada (simulação)
cliente = "Empresa Alpha"
cod_plano = 202
tipo_cliente = "PJ"

# Uso das funções
plano = nome_plano(cod_plano)
total = calcular_total_fatura(cod_plano, tipo_cliente)

# Saída do sistema
if total is None:
      print("Erro ao gerar fatura. Verifique os dados informados.")
else:
      print("Cliente:", cliente)
      print("Plano contratado:", plano, f"(código {cod_plano})")
      print(f"Total da fatura: R$ {total:.2f}")
```

Esse trecho representa o programa principal, responsável por utilizar as funções criadas anteriormente para gerar e exibir as informações da fatura de forma organizada e compreensível.

Inicialmente, são definidos os dados que simulam um cenário real de faturamento: o nome do cliente, o código do plano contratado e o tipo de cliente. Esses valores seriam, em um sistema real, obtidos a partir de um cadastro ou entrada de dados.

Em seguida, a função nome_plano é chamada para converter o código do plano em um nome legível, permitindo que a informação seja apresentada ao usuário de forma clara, sem expor detalhes técnicos internos do sistema.

Logo após, a função calcular_total_fatura é utilizada para calcular o valor total da fatura. O resultado retornado por essa função é armazenado na variável total, possibilitando seu uso em decisões posteriores.

A estrutura condicional verifica se o valor retornado é None. Esse teste indica se ocorreu algum problema durante o cálculo, como um código de plano ou tipo de cliente inválido. Caso isso aconteça, o sistema exibe uma mensagem de erro orientando a verificação dos dados informados.

Se o cálculo for bem-sucedido, o programa exibe os dados da fatura de forma organizada: o nome do cliente, o plano contratado (com nome e código) e o valor total da fatura, formatado como valor monetário.

Checklist — Revisão deste exercício (tudo OK)

Com base nas funções e no programa principal do módulo de faturamento, todos os itens abaixo foram atendidos:

- ✅ O nome da função indica claramente o que ela faz?
- ✅ A função executa apenas uma tarefa?
- ✅ O tamanho da função está adequado e fácil de entender?
- ✅ Os parâmetros são realmente necessários?
- ✅ O uso de return está correto quando o valor precisa ser reutilizado?
- ✅ O código está organizado e legível?

Este exercício simula uma situação comum em sistemas financeiros profissionais, nos quais a clareza das regras de negócio, a organização do código e a separação de responsabilidades são fundamentais. A estrutura adotada demonstra como funções bem definidas contribuem para soluções mais seguras, fáceis de manter e simples de testar.



Uso excessivo

Embora o uso de funções seja essencial para organizar e reutilizar código, é importante compreender que exagerar na fragmentação também pode gerar complexidade desnecessária. Criar funções para tarefas muito simples ou excessivamente específicas pode dificultar a leitura do código, tornando o fluxo do programa menos intuitivo.

O objetivo das funções não é dividir o código ao máximo, mas organizar responsabilidades de forma coerente. Uma boa função deve representar uma ação clara e significativa dentro do sistema. Quando a divisão é feita com equilíbrio, o código se torna mais legível e fácil de manter; quando feita em excesso, pode exigir que o leitor navegue por muitas funções para entender um processo simples.

Portanto, ao criar funções, avalie sempre o contexto e o propósito. O ideal é encontrar um ponto de equilíbrio entre organização e simplicidade, garantindo que o código permaneça claro, eficiente e compreensível.